

Analysis errored

Python

```
from docx import Document
from docx.shared import Inches, Pt
from docx.enum.text import WD_ALIGN_PARAGRAPH
from docx.enum.table import WD_TABLE_ALIGNMENT, WD_CELL_VERTICAL_ALIGNMENT
from pathlib import Path
import re

docx_path = Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.docx")
md_path = Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.md")

content = r"""# TIZIM_02 – GLOBAL CONSTRUCTION OS
## Yakuniy arxitektura va rivojlantirish rejasi
### Supabase/PostgreSQL Core + Cloudflare + Google Sheets Full Client + Google Drive + AI

**Hujjat maqomi:** Canonical target architecture / master roadmap
**Asosiy prinsip:** Supabase – yagona Source of Truth; Cloudflare Web – to'liq klient; Go
**Migration modeli:** Tizim_01 ishlaydigan reference/rollback; Tizim_02 bosqichma-bosqich
**Security:** hozir architecture security-ready bo'ladi; to'liq security hardening yakuni
**Muhim:** bu hujjat "hammasini hozir qurish" emas. U yakuniy maqsad va qat'iy qurilish t

---

# 1. VISION – NIMA QURILADI?

Maqsad oddiy "smeta dasturi" yoki oddiy ERP yaratish emas.

Yakuniy mahsulot:

> **Construction Operating System (Construction OS)** – qurilish kompaniyasi va qurilish

Tizim quyidagilarni birlashtirishi kerak:

- Smeta
- Narxlash
- LRV / nakopitelniy
- FAKT
- F2 / progress certification
- Akt / AOSR / APPOK
- Shartnomalar
- Xarid / Procurement
- Sklad
- Yetkazib beruvchilar
- To'lovlar
- Debitor/kreditor
- Didox / EDO
- Bank integratsiyalari
```

- Loyiha hujjatlari
- BIM/CAD/project takeoff
- Turar joy yoki boshqa unit-based sotuv
- CRM
- Dashboard
- Audit
- AI data query
- RAG / knowledge
- AI agents
- Monitoring / anomaly detection
- Performance/self-improvement
- Global localization

2. USERNING YAKUNIY ARXITEKTURA TALABI

Bir kompaniya turli loyihalarda turli rolga ega bo'lishi mumkin:

- bitta loyiha – Buyurtmachi;
- boshqa loyiha – Pudratchi;
- boshqa loyiha – Loyihachi;
- boshqa loyiha – Yetkazib beruvchi.

Shuning uchun rol ****Company-level global atribut emas****.

To'g'ri model:

```text

Company

↓

Project Participation

↓

Role in this Project/Contract

Misol:

OUR COMPANY

├─ Project A → CONTRACTOR

├─ Project B → OWNER

├─ Project C → DESIGNER

└─ Project D → SUPPLIER

Bu model butun platforma uchun fundamental qoida.

## 3. TARGET ARCHITECTURE

USERS

|

## 4. ROLELAR VA WORKSPACE

Bitta frontend bo'лади.

Har bir foydalanuvchi rolga va project participation'iga qarab workspace ko'radi.

## Masalan:

# Buyurtmachi

- loyiha holati
- byudjet
- progress
- F2
- to'lov
- hujjatlar
- risk

## **Pudratchi**

- smeta
- FAKT
- F2
- sklad
- procurement
- shartnoma
- xarajat
- hujjatlar

## **Loyihachi**

- chizmalar
- revision
- RFI
- BIM
- approval
- takeoff

## **Yetkazib beruvchi**

- purchase order
- yetkazib berish
- invoice
- payment status
- documents

## **Direktor/CFO**

- barcha loyihalar
- cashflow
- debitor/kreditor
- profit
- risk
- anomaly
- AI summary

**Alohida 10 ta frontend qurilmaydi.** Bitta platforma ichida role-based workspace quriladi.

---

## **5. SUPABASE — SINGLE SOURCE OF TRUTH**

Supabase/PostgreSQL authoritative hisoblanadi.

Unda business state saqlanadi:

- company
- users
- memberships
- projects
- project participants
- objects
- estimates
- estimate rows
- resources
- price catalog
- contracts
- purchase
- warehouse
- facts
- F2
- acts
- invoices
- payments
- bank transactions
- sales
- documents metadata
- audit
- versions
- sync state
- conflicts
- AI knowledge metadata

Supabase “cache” emas.

---

## 6. GOOGLE SHEETS — TO‘LIQ TENG KLIENT

Sheets:

- o‘qiydi;
- tahrirlaydi;
- ruxsatli write operationlar bajaradi;
- F2/FAKT bilan ishlaydi;
- export/print/Excel workflow beradi.

Lekin authoritative calculation Sheets formulasi bo‘lmaydi.

To'g'ri oqim:

```
Sheets
↓
GAS adapter
↓
Supabase API/RPC
↓
PostgreSQL
↓
result
↓
Sheets
```

Sheets foydalanuvchisi ham web foydalanuvchisi bilan **bir xil backend operationni** bajaradi.

---

## 7. GOOGLE DRIVE — ORIGINAL HUJJATLAR OMBORI

Drive:

- original PDF
- Excel
- DWG
- RVT
- IFC
- shartnoma
- invoice
- F2
- akt
- AOSR
- APPOK
- scan
- loyiha fayllari

uchun document store.

Supabase:

```
document_id
entity_type
entity_id
drive_file_id
file_name
mime_type
```

```
file_hash
version
created_by
created_at
```

saqlaydi.

Drive faylining o'zi ERP truth emas.

## 8. DATA MODEL — CANONICAL FOUNDATION

Minimum entity hierarchy:

```
Company
├── Project
│ ├── Object
│ │ ├── Estimate
│ │ │ └── Estimate Row
│ │ ├── Fact
│ │ ├── F2
│ │ ├── Contract
│ │ ├── Purchase
│ │ ├── Warehouse
│ │ └── Documents
│ ├── Participants
│ ├── Contracts
│ ├── Finance
│ └── Sales/Units
```

Tavsiya etiladigan domain jadvallar:

```
companies
users
memberships
projects
project_participants
objects
estimates
estimate_rows
estimate_sections
resources
materials
material_aliases
price_catalog
price_history
contracts
contract_items
purchase_orders
```

```
purchase_items
warehouse_documents
warehouse_rows
stock_balances
facts
fact_rows
f2_documents
f2_rows
acts
invoices
payments
bank_accounts
bank_transactions
sales_projects
buildings
units
reservations
sales_contracts
sales_payments
documents
document_links
audit_log
field_changes
sync_state
sync_conflicts
ai_query_log
ai_source_log
```

**T2** jadvallari migration/canonical estimate layer sifatida ishlatilishi mumkin; keyinchalik domain-specific jadvallarga evolyutsiya qilinadi.

---

## 9. IMMUTABLE ID

Sheets row number yoki Excel koordinatasi primary key bo'lmaydi.

Har entity:

```
id = immutable identifier
```

bo'ladi.

Sheets'da yashirin:

```
SUPABASE_ID
VERSION
```

bo'lishi kerak.



Sorting/filtering/row insert/delete entity identityni buzmasligi kerak.

---

## 10. VERSIONING VA CONCURRENCY

Full two-way sync uchun:

```
id
version
updated_at
updated_by
updated_source
```

majburiy.

Update:

SQL

```
UPDATE ...
SET version = version + 1
WHERE id = :id
AND version = :expected_version;
```

0 rows updated:

CONFLICT

Last-write-wins moliyaviy ma'lumotlarda ishlatilmaydi.

Conflict response server state + client attempt + versionni beradi.

---

## 11. AUDIT VA FIELD HISTORY

Har muhim mutation:

```
who
when
source
entity
entity_id
operation
field
old_value
new_value
```

```
request_id
operation_id
```

bilan kuzatiladi.

Source:

```
frontend
sheets
gas
import
system
ai
```

bo'lishi mumkin.

---

## 12. IDEMPOTENCY

Har transaction uchun kerak bo'lgan joyda:

```
operation_id = UUID
```

bo'ladi.

Aynan request qayta kelganida ikkinchi marta hujjat yoki pul operatsiyasi yaratilmaydi.

Ayniqsa:

- F2
- Fact
- purchase
- bank import
- Didox import
- webhook
- payment

uchun majburiy.

---

## 13. SMETA ENGINE

Tizim\_02 quyidagicha ishlaydi:

```
ORIGINAL FILE
↓
```

```
RAW IMPORT
↓
NORMALIZATION
↓
CLASSIFICATION
↓
PRICING
↓
ROLLUP
↓
CANONICAL ESTIMATE
```

`T2_Import.js` bu yoʻnalishda mavjud poydevor sifatida saqlanadi.

Raw layer oʻzgarmas boʻladi.

## 14. PRICING ENGINE

Prioritet:

1. fayldagi tayyor narx;
2. exact code + unit;
3. normalized name + unit;
4. alias;
5. fuzzy/trigram;
6. manual review;
7. AI candidate.

AI narxni “oʻylab topmaydi”.

AI faqat:

```
candidate
confidence
source
```

beradi.

Final authoritative price deterministic qoidalar/RPC orqali yoziladi.

## 15. 20 000+ QATOR PERFORMANCE

Bunday ishlarda:

```
20 000 rows
x
```

```
Sheets API
x
formula/merge
```

taqiqlanadi.

PostgreSQL set-based calculation:

```
20 000 rows
↓
JOIN
↓
INDEX
↓
AGGREGATION
↓
RPC
```

bo'ladi.

Maqsad:

- 1k;
- 5k;
- 20k;
- 50k

benchmark.

P50/P95 latency, throughput va concurrent users o'lchanadi.

"0.05 s bo'ladi" kabi kafolat faqat real benchmarkdan keyin beriladi.

---

## 16. F2 + FAKT + NAKOPITELNIY

Bu tizimning moliyaviy yuragi.

Bog'lanish:

```
Estimate Row
↑
| immutable ID
├─ Fact Row
└─ F2 Row
```

Server-side:

```
Estimate
→ Fact
→ F2
→ Remaining
```

hisoblanadi.

Nakopitelniy qayta-qayta Sheetsni to'liq o'qimaydi.

F2/Fakt mutationlari domain transaction bo'ladi.

---

## 17. FINANCE MODEL

Bitta "price" yoki "amount" hamma narsani ifodalamasin.

Tushunchalar:

```
estimate_price
contract_price
supplier_price
purchase_price
invoice_amount
actual_cost
market_price
sales_price
payment_amount
```

Pul oqimi:

```
Contract
↓
Invoice
↓
Payment
```

Alohida.

Accounting state ham alohida.

---

## 18. PROCUREMENT + WAREHOUSE

```
Estimate
↓
Material Requirement
```

↓  
Purchase  
↓  
Supplier  
↓  
Invoice  
↓  
Warehouse Receipt  
↓  
Stock  
↓  
Issue/Consumption  
↓  
Fact

Shu zanjir orqali:

“M200 qancha kerak, qancha sotib olindi, qancha keldi, qancha ishlatildi, qancha qoldi?”

bitta graphdan javoblanadi.

## 19. PROJECT / BIM / TAKEOFF

Kelajak:

DWG  
PDF  
IFC  
RVT  
Excel  
↓  
Extraction  
↓  
Normalized quantities  
↓  
Project Takeoff  
↓  
Resource Mapping  
↓  
Estimate Comparison

Misol:

Project = 1284 m<sup>3</sup> concrete  
Estimate = 1190 m<sup>3</sup>  
Variance = +94 m<sup>3</sup>

AI sabab va riskni tushuntiradi; source quantity provenance saqlanadi.

---

## 20. DOCUMENT INTELLIGENCE

Drive hujjatlari:

```
PDF
image
Excel
scan
```

OCR/parser orqali structured metadata va candidate rowsga aylantiriladi.

AI output:

```
document_type
supplier
contract
invoice_no
date
items
amount
currency
tax
confidence
```

bo'ladi.

Final write deterministic validation orqali.

---

## 21. DIDOX / EDO

Didox platformaga adapter sifatida ulanadi.

Core:

```
EDO Adapter Interface
```

Country/provider:

```
Uzbekistan → Didox
```

Integratsiya:

```
Counterparty
Contract
EHF/Invoice
Document State
Amount
VAT
External ID
```

Duplicate imports idempotent bo'ladi.

---

## 22. BANK INTEGRATION

Bank core:

```
Bank Adapter Interface
```

Har bank:

```
BankProviderAdapter
```

Bank transaction:

```
account
transaction_id
date
amount
currency
counterparty
description
reference
```

Reconciliation:

```
Bank Transaction
↓
Candidate Match
↓
Invoice / Payment / Contract
↓
Reconciled / Unmatched
```

AI candidate topishi mumkin, lekin avtomatik moliyaviy truthga aylantirish business rule orqali bo'ladi.

---



## 23. SALES / REAL ESTATE

Agar loyiha turar joy yoki unit-based asset sotadigan bo'lsa:

```
Project
↓
Building
↓
Section
↓
Floor
↓
Unit
↓
Reservation
↓
Sales Contract
↓
Payment Schedule
↓
Payment
```

Unit status:

```
available
reserved
sold
cancelled
```

Project profitability bilan bog'lanadi.

---

## 24. MULTI-ROLE / MULTI-TENANT

Company global rolega ega emas.

Role project participation/contract scope bilan belgilanadi.

Keyinchalik multi-tenant SaaS:

```
Company A
Company B
Company C
```

bir-biridan izolyatsiya qilinadi.

Hozir bitta kompaniya bilan ishlash mumkin, lekin data model company boundaryni buzadigan hard-code qilmasin.

## 25. GLOBALIZATION

Core global bo'ladi.

Country-specific pack:

```
Uzbekistan Pack
UAE Pack
Saudi Pack
USA Pack
EU Pack
UK Pack
...
```

Country layer:

- tax;
- EDO;
- accounting rules;
- construction standards;
- terminology;
- currency;
- units;
- document requirements;
- signature;
- banking;
- government integrations.

Core'da Uzbekistan-specific conditionlar hard-code qilinmaydi.

---

## 26. LANGUAGE / LOCALE

Locale layer:

```
language
date_format
number_format
currency
timezone
units
terminology
```

Terminology dictionary kerak.

Masalan:

```
F2
Payment Application
Progress Claim
Work Certificate
```

bir xil core conceptga mapping qilinishi mumkin.

## 27. AI DATA LAYER

Hozirgi `66_AI_Data.js` dagi konsepsiya keyinchalik quyidagiga evolyutsiya qiladi:

```
USER QUESTION
↓
AI ORCHESTRATOR
↓
INTENT
↓
DATA TOOL / SQL TOOL / DOCUMENT TOOL
↓
SUPABASE
↓
VERIFIED EVIDENCE
↓
AI ANSWER
```

20 000 qatorni LLM contextiga tiqish kerak emas.

## 28. “M200 QANCHA OLINGAN?” SAVOLI

To'g'ri ishlash:

```
User:
M200 beton qancha olingan?

AI:
material = M200
metric = purchased_quantity
scope = current project/object
↓
get_material_purchase_summary()
↓
Postgres
↓
```

1284 m<sup>3</sup>



AI response

AI:

“M200 beton 1 284 m<sup>3</sup> olingan.”

deydi.

Agar qo'shimcha savol:

“Qancha qoldi?”

AI:

estimate  
purchase  
fact  
warehouse

tool'larini chaqirib, hisoblangan natija beradi.

---

## 29. AI MEMORY VA KNOWLEDGE

AI xotirasi uch qatlam:

STRUCTURED DATA → PostgreSQL  
DOCUMENTS → Drive + indexed chunks  
KNOWLEDGE → vector/RAG layer

Conversation context alohida.

AI “topilmadi” deyishdan oldin:

1. exact;
2. alias;
3. semantic;
4. related tables;
5. documents

qidiradi.

---

## 30. AGENT ARXITEKTURASI

Bitta ulkan “JARVIS” agent emas.

Markazda:

AI ORCHESTRATOR

bor.

Specialized agents:

Monitoring Agent  
Estimate Agent  
F2/Fact Agent  
Procurement Agent  
Finance Agent  
Warehouse Agent  
Document Agent  
Project/BIM Agent  
Sales Agent  
Audit Agent  
Performance Agent  
Code/Architecture Agent

## 31. AGENT TOOL MODEL

Agent:

```
get_project()
get_estimate()
get_material_summary()
get_purchase_summary()
get_fact()
get_f2()
get_balance()
get_price_history()
search_documents()
create_f2_draft()
create_purchase_draft()
send_notification()
```

kabi oldindan ruxsat etilgan tools bilan ishlaydi.

Agentga:

```
generic SQL
generic delete
```

berilmaydi.

---

## 32. AGENTLARNING AVTONOMIYASI

3 daraja:

### L1 — READ

O'qiydi, tahlil qiladi, xabar beradi.

### L2 — ACTION

Draft/controlled mutation qiladi.

### L3 — AUTONOMOUS

Schedule asosida o'zi tekshiradi va belgilangan ishlarni bajaradi.

Real tashqi ta'sirli operations uchun approval/HITL.

---

## 33. SELF-IMPROVING SYSTEM

Kelajak agent loop:

```
OBSERVE
↓
DETECT
↓
ANALYZE
↓
PROPOSE
↓
SANDBOX
↓
TEST
↓
COMPARE
↓
APPROVAL
↓
DEPLOY
↓
MONITOR
```

Agent production kodini o'zicha almashtirmaydi.

Performance agent:

```
slow query
↓
EXPLAIN
↓
candidate index
↓
benchmark
↓
proposal
```

qiladi.

---

## 34. CONTROL CENTER

Dastlab alohida frontend shart emas.

Mavjud frontend ichida:

```
/admin
```

bo'ladi.

Keyinchalik alohida:

```
admin.domain.com
```

qilib ajratish mumkin.

Control Center:

```
Companies
Users
Projects
Integrations
AI
Agents
Jobs
Sync
Conflicts
Errors
Performance
Documents
Audit
System Health
```

ni boshqaradi.

Direktor uchun esa alohida management dashboard:

```
Project Health
Financial Health
Procurement Risk
Schedule Risk
Document Risk
AI Alerts
```

bo'ladi.

---

## 35. JOBS / QUEUES / WORKFLOWS

Uzoq ishlar:

- katta import;
- document parsing;
- AI OCR;
- bank import;
- Didox sync;
- Drive indexing;
- report generation;
- large reconciliation;

request ichida bajarilmaydi.

```
REQUEST
↓
JOB
↓
QUEUE/WORKFLOW
↓
WORKER
↓
PROGRESS
↓
RESULT
```

Retry/idempotency bo'ladi.

---

## 36. FRONTEND PERFORMANCE



20k qatorni browserga birdan yubormang.

Talab:

- server-side pagination;
- filtering;
- sorting;
- virtualization;
- lazy tree expansion;
- selective columns;
- server-side aggregates;
- debounced search.

Mobil ham benchmark qilinadi.

---

## 37. GOOGLE SHEETS PERFORMANCE

Sheets clientda:

- batch read/write;
- batch RPC;
- debounce;
- no per-cell API call;
- no full rebuild unless requested;
- incremental sync.

Full rebuild recovery uchun.

Normal ishlash incremental.

---

## 38. MIGRATION ROADMAP

### FAZA 0 — BASELINE

- Tizim\_01 snapshot.
- Golden datasets.
- Existing totals.
- Rollback.
- T2 isolation.

### FAZA 1 — T2 CORE

- raw layer;

- normalization;
- classification;
- pricing;
- rollup.

## **FAZA 2 — REAL DATA**

- Amfiteatr 4 937 qator;
- real objects;
- full reconciliation.

## **FAZA 3 — VERSION/CONFLICT**

- version;
- audit;
- conflict;
- idempotency.

## **FAZA 4 — FRONTEND FULL CLIENT**

- read;
- write;
- conflict handling.

## **FAZA 5 — SHEETS FULL CLIENT**

- `ИШЧИ СМЕТА` ;
- hidden ID/version;
- two-way sync;
- conflict handling.

## **FAZA 6 — FAKT/F2/NAKOPITELNIY**

- canonical financial chain;
- transaction;
- reconciliation.

## **FAZA 7 — PROCUREMENT/Warehouse**

## **FAZA 8 — DOCUMENTS/DRIVE**

## **FAZA 9 — DIDOX/BANK**

## **FAZA 10 — SALES**

## **FAZA 11 — BIM/TAKEOFF**

**FAZA 12 — AI DATA LAYER**

**FAZA 13 — AI AGENTS**

**FAZA 14 — SELF-IMPROVEMENT**

**FAZA 15 — SECURITY HARDENING**

**FAZA 16 — GLOBALIZATION / COUNTRY PACKS**

**FAZA 17 — PRODUCTION CUTOVER**

---

## **39. SECURITY STRATEGIYASI**

Foydalanuvchi talabi:

eng katta security ishlarini oxirgi bosqichda qilish.

Qabul qilinadi.

Lekin hozirdanoq:

- secrets source code'da yo'q;
- service role clientga chiqmaydi;
- generic SQL endpoint yo'q;
- write actions RPC/domain orqali;
- immutable ID;
- version;
- audit fields;
- operation\_id;
- source/origin metadata

architectureda saqlanadi.

Final stage:

- RLS;
- full RBAC/ABAC;
- SSO/MFA;
- secret rotation;
- API abuse protection;
- rate limit;
- webhook signatures;
- audit integrity;

- backup/restore;
  - penetration/security regression;
  - tenant isolation;
  - data residency.
- 

## 40. TEST STRATEGY

### Golden data

- kichik;
- o'rta;
- Amfiteatr 4 937;
- 20k+ stress.

### Unit

Normalization, classification, pricing, calculation.

### DB

RPC, transaction, constraint, version, idempotency.

### Integration

Frontend ↔ API ↔ DB.

Sheets ↔ GAS ↔ DB.

Drive.

Didox.

Bank.

### Reconciliation

Tizim\_01 ↔ Tizim\_02:

- row count;
  - quantity;
  - price;
  - category;
  - total;
  - F2;
  - Fact;
  - remaining;
  - special rows.
-

## 41. DEFINITION OF DONE

Tizim\_02:

- Supabase canonical;
- Frontend full client;
- Sheets full client;
- Drive linked;
- ID/version/conflict;
- audit;
- idempotency;
- F2/Fact server-side;
- procurement;
- warehouse;
- document layer;
- 4,937 golden reconciliation;
- 20k benchmark;
- AI data layer;
- agent layer;
- control center;
- rollback;
- final security hardening

bitmaguncha “production-ready” deyilmaydi.

---

## 42. CLAUDE CODE ISH TARTIBI

Claude har task oldidan:

1. Repository va mavjud domain logicni o'qiydi.
2. `00_BOSH_QONUN.md` va Tizim\_02 master hujjatini o'qiydi.
3. Existing RPC/function/adapterni qidiradi.
4. Dependency/data-flowni aniqlaydi.
5. Minimal o'zgarish rejasini beradi.
6. Kod qiladi.
7. Test qiladi.
8. Regression qiladi.
9. Kerak bo'lsa Tizim\_01 bilan solishtiradi.
10. Hisobot beradi.

Task tugamaguncha keyingi katta featurega o'zboshimcha sakramaydi.

---

## 43. CLAUDE QILMASLIGI KERAK

- Tizim\_01ni migration tajribasi uchun buzmaslik.
- Existing functionni topmasdan parallel function yaratmaslik.
- Bir xil business logicni uch joyda yozmaslik.
- Sheets formulasini authoritative calculation qilishni davom ettirmaslik.
- Row numberni entity ID deb ishlatmaslik.
- Version checkni chetlab o'tmaslik.
- Last-write-winsni moliyaviy data uchun ishlatmaslik.
- Generic SQL/write endpoint yaratmaslik.
- Alga moliyaviy raqamni o'ylab topdirib qo'yimaslik.
- Fake data qo'shmaslik.
- Benchmark qilmasdan performance claim bermaslik.
- Migration farqlari turib keyingi critical modulga o'tmaslik.
- Security architecture talablarini buzmaslik.
- Hozir full security hardeningni developmentni keraksiz sekinlashtirish darajasida oldinga surmaslik.

## 44. HOZIRGI ENG MUHIM ISH

Barcha katta orzularni vaqtincha chetga qo'yib, birinchi production foundation:

```
T2 Core
↓
Amfiteatr 4937
↓
Tizim_01 vs Tizim_02
↓
Version
↓
Frontend write
↓
Sheets write
↓
F2/Fact/Nakopitelniy
```

ketma-ketligi.

**Didox, bank, BIM, sales, autonomous agents — arxitekturada joyi oldindan belgilanadi, lekin core correctnessdan oldin qurilmaydi.**

## 45. STRATEGIK YAKUNIY QAROR

## Platforma:

```
SUPABASE
= MIYA / SOURCE OF TRUTH

CLOUDFLARE
= WEB + API + AGENTS

GOOGLE SHEETS
= FULL ALTERNATIVE CLIENT

GOOGLE DRIVE
= ORIGINAL DOCUMENT STORE

AI
= INTELLIGENCE LAYER

AGENTS
= AUTOMATION / MONITORING / ORCHESTRATION

COUNTRY PACKS
= LOCAL LAW/EDO/TAX/ACCOUNTING/TERMINOLOGY
```

Bir kompaniya bir vaqtning o'zida turli loyihalarda turli rollarga ega bo'lishi mumkin.

Bitta frontend ichida role-based workspace ishlaydi.

Bitta project graph:

```
Estimate
Fact
F2
Purchase
Warehouse
Contract
Invoice
Payment
Documents
Sales
```

hammasini bog'laydi.

Global architecture O'zbekiston-specific emas; O'zbekiston — birinchi country implementation.

---

## 46. THE NORTH STAR

Foydalanuvchi istalgan interfeysdan:

Google Sheets  
Web  
Mobile  
Telegram

ishlaydi.

Tizim esa bitta markaziy ma'lumot modelidan foydalanadi.

Foydalanuvchi:

“M200 qancha olingan?”

desa — database'dan topadi.

“Qancha qoldi?”

desa — estimate + fact + warehouse + F2ni birlashtiradi.

“Kimdan olingan?”

desa — supplier + invoice + contractni bog'laydi.

“Qancha to'ladik?”

desa — payment/bank bilan reconciliation qiladi.

“Bu loyihada risk nima?”

desa — project, finance, procurement, schedule va document signallarini birlashtiradi.

“Bugun nima qilish kerak?”

desa — agentlar priority/action plan chiqaradi.

Va tizim vaqt o'tishi bilan:

observe  
→ learn  
→ detect  
→ propose  
→ test  
→ improve

siklida rivojlanadi — lekin authoritative data va production o'zgarishlari nazorat qilinadigan mexanizmlar orqali.



## 47. BIRINCHI IJROCHI TASK

**Claude Code'ning keyingi sessiyasini faqat shu bilan boshlash:**

"TIZIM\_02 target architecture'ni canonical deb qabul qil. Avval repositorydagi barcha T2/Supabase/Sheets bridge/Frontend API/F2/Fact/AI qismlarini map qil. Yangi feature yozma. Qaysi core contracts allaqachon bor, qaysilari yetishmaydi, qaysilari bir-biriga zid — jadval bilan chiqar. Keyin Amfiteatr 4,937 qatorli golden migration/reconciliation uchun aniq execution plan ber. Farqlar yopilmaguncha yangi critical module yozma."

Bu hujjatning birinchi ijro gate'idir.

""

```
md_path.write_text(content, encoding="utf-8")
```

```
doc = Document()
```

```
sec = doc.sections[0]
```

```
sec.top_margin = Inches(0.65)
```

```
sec.bottom_margin = Inches(0.65)
```

```
sec.left_margin = Inches(0.7)
```

```
sec.right_margin = Inches(0.7)
```

```
styles = doc.styles
```

```
styles["Normal"].font.name = "Aptos"
```

```
styles["Normal"].font.size = Pt(9.5)
```

```
styles["Title"].font.name = "Aptos Display"
```

```
styles["Title"].font.size = Pt(22)
```

```
styles["Heading 1"].font.name = "Aptos Display"
```

```
styles["Heading 1"].font.size = Pt(15)
```

```
p = doc.add_paragraph()
```

```
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
```

```
r = p.add_run("TIZIM_02 — GLOBAL CONSTRUCTION OS")
```

```
r.bold = True
```

```
r.font.size = Pt(22)
```

```
p = doc.add_paragraph()
```

```
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
```

```
r = p.add_run("Yakuniy arxitektura va rivojlantirish rejasi")
```

```
r.italic = True
```

```
r.font.size = Pt(12)
```

```
p = doc.add_paragraph()
```

```
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
```

```
p.add_run("Supabase/PostgreSQL Core • Cloudflare • Sheets Full Client • Drive •
AI/Agents • Global Country Packs").font.size = Pt(9)
```

```
for block in content.split("\n\n"):
 block = block.strip()
 if not block:
 continue
 if block.startswith("# "):
 doc.add_heading(block[2:].strip(), level=0)
 elif block.startswith("## "):
 doc.add_heading(block[3:].strip(), level=1)
 elif block.startswith("### "):
 doc.add_heading(block[4:].strip(), level=2)
 elif block.startswith(">"):
 for line in block.splitlines():
 p = doc.add_paragraph()
 p.paragraph_format.left_indent = Inches(0.25)
 rr = p.add_run(line)
 rr.italic = True
 else:
 # preserve code-ish blocks as monospace
 if block.startswith(" ") and block.endswith(" "):
 code = block[3:-3].strip("\n")
 p = doc.add_paragraph()
 p.paragraph_format.left_indent = Inches(0.2)
 r = p.add_run(code)
 r.font.name = "Consolas"
 r.font.size = Pt(8)
 else:
 for line in block.splitlines():
 line = line.rstrip()
 if not line:
 continue
 if line.startswith("- "):
 p = doc.add_paragraph(line[2:], style="List Bullet")
 elif re.match(r"^\d+\.s", line):
 p = doc.add_paragraph(re.sub(r"^\d+\.s", "", line), style="List Number")
 else:
 p = doc.add_paragraph(line)
```

```

for section in doc.sections:
 header = section.header.paragraphs[0]
 header.text = "TIZIM_02 — GLOBAL CONSTRUCTION OS | MASTER ARCHITECTURE"
 header.runs[0].font.size = Pt(8)
 footer = section.footer.paragraphs[0]
 footer.alignment = WD_ALIGN_PARAGRAPH.CENTER
 footer.add_run("Canonical target architecture • Supabase Core • Two full clients •
 AI/Agents").font.size = Pt(8)

doc.save(docx_path)

print(docx_path)
print(md_path)

```

Analysis errored

### Python

```

from pathlib import Path
from docx import Document
from docx.shared import Inches, Pt
from docx.enum.text import WD_ALIGN_PARAGRAPH

md = r"""# TIZIM_02 — GLOBAL CONSTRUCTION OS
Yakuniy arxitektura va rivojlantirish rejasi

Canonical maqsad: Supabase/PostgreSQL — yagona miya va Source of Truth; Cloudflare —

Migration prinsipi: Tizim_01 ishlaydigan reference/rollback sifatida saqlanadi. Tizim

Security prinsipi: to'liq security hardening oxirgi katta bosqichda bajariladi. Biroq

1. YAKUNIY MAHSULOT

Bu loyiha "smeta dasturi" emas. Yakuniy mahsulot — qurilish kompaniyasi va qurilish loyih

- Smeta va narxlash
- LRV/nakopitelniy
- Fakt
- F2 / progress certification
- Akt/AOSR/APPOK
- Shartnomalar

```



### 3. ROLLAR: KOMPANIYA QAYERDA QANDAY ROLDA?

Kompaniyaning bitta global roli bo'lmaydi.

```
Company A
├─ Project A → CONTRACTOR
├─ Project B → OWNER
├─ Project C → DESIGNER
└─ Project D → SUPPLIER
```

Demak rol:

**Company → Project Participation → Contractual Role**

darajasida belgilanadi.

Bitta kompaniya bir vaqtning o'zida turli loyihalarda turli rolga ega bo'lishi mumkin.

## 4. KLIENT MODELI

### 4.1 Google Sheets

Sheets oddiy "export" emas.

U:

- o'qiydi;
- ruxsatli maydonlarni tahrirlaydi;
- F2/Fakt bilan ishlaydi;
- Supabase orqali saqlaydi;
- sync oladi;
- Excel/print workflow beradi.

### 4.2 Cloudflare Frontend

Web:

- o'qiydi;
- yozadi;
- dashboard;
- workflow;

- AI;
- hujjat;
- project control.

**Sheets va frontend bir xil core operationlardan foydalanadi.**

```
Frontend → Worker → RPC → Postgres
Sheets → GAS adapter → RPC → Postgres
```

Business logic ikki joyda takrorlanmaydi.

---

## 5. GOOGLE DRIVE

Drive — original binary/document store:

- PDF
- Excel
- DWG
- RVT
- IFC
- F2
- Akt
- AOSR
- APPOK
- Invoice
- Shartnoma
- Scan

Supabase hujjat metadata'sini saqlaydi:

```
document_id
entity_type
entity_id
drive_file_id
file_name
mime_type
hash
version
created_by
created_at
```

Drive faylining o'zi ERP truth emas.

---

## 6. SUPABASE CORE

Canonical domain:

```
companies
users
memberships
projects
project_participants
objects

estimates
estimate_sections
estimate_rows
resources
materials
material_aliases
price_catalog
price_history

facts
fact_rows
f2_documents
f2_rows
acts

contracts
contract_items
purchase_orders
purchase_items

warehouse_documents
warehouse_rows
stock_balances

invoices
payments
bank_accounts
bank_transactions

sales_projects
buildings
units
reservations
sales_contracts
sales_payments

documents
document_links

audit_log
```

```
field_changes
sync_state
sync_conflicts
ai_query_log
ai_source_log
```

T2 jadvallari migration/canonical estimate layer sifatida ishlashi mumkin; keyinchalik domain-specific modelga evolyutsiya qilinadi.

---

## 7. IMMUTABLE ID

Sheets row number authoritative ID bo'lmaydi.

Har entity:

```
id
```

bilan aniqlanadi.

Sheets'da yashirin:

```
SUPABASE_ID
VERSION
```

bo'ladi.

Sort, filter, insert/delete identityni buzmasligi kerak.

---

## 8. VERSIONING VA CONCURRENCY

Canonical metadata:

```
id
version
updated_at
updated_by
updated_source
```

Update:

```
SQL
```

```
UPDATE ...
SET version = version + 1
```



```
WHERE id = :id
```

Agar 0 row update bo'lsa:

```
CONFLICT
```

Moliyaviy ma'lumotlarda silent Last-Write-Wins ishlatilmaydi.

Conflict response:

```
entity_id
expected_version
actual_version
server_state
client_attempt
```

---

## 9. AUDIT + IDEMPOTENCY

Har muhim mutation:

```
who
when
source
operation
entity
entity_id
old_value
new_value
request_id
operation_id
```

saqlaydi.

Source:

```
frontend
sheets
gas
import
system
ai
```

bo'lishi mumkin.

Muhim mutationlarda `operation_id` duplicate requestni qayta transaction qilmaslik uchun ishlatiladi.

## 10. SMETA ENGINE

Target:

```
ORIGINAL FILE
→ RAW IMPORT
→ NORMALIZATION
→ CLASSIFICATION
→ PRICING
→ ROLLUP
→ CANONICAL ESTIMATE
```

`T2_Import.js` poydevori saqlanadi.

Raw layer o'zgarmas bo'ladi.

---

## 11. PRICING ENGINE

Prioritet:

1. fayldagi tayyor narx;
2. exact code + unit;
3. normalized name + unit;
4. alias;
5. fuzzy/trigram;
6. manual review;
7. AI candidate.

AI production narxni ixtiro qilmaydi.

AI:

```
candidate
confidence
source
```

beradi; final write deterministic rule/RPC orqali amalga oshadi.

---

## 12. 20 000+ QATOR PERFORMANCE

Quyidagidan qochiladi:

```
20 000 rows
x
Sheets API
x
per-row formula/merge
```

Target:

```
20 000 rows
→ PostgreSQL
→ indexed JOIN
→ aggregation
→ RPC
→ result
```

Benchmark:

- 1k
- 5k
- 20k
- 50k

uchun:

- import
- classification
- exact pricing
- fuzzy pricing
- rollup
- F2 aggregation
- API
- frontend

o'lchanadi.

P50/P95 latency va concurrency yoziladi.

---

## 13. FAKT + F2 + NAKOPITELNIY

Canonical relation:

```
Estimate Row
├─ Fact Row
└─ F2 Row
```

ID orqali bog'lanadi.

Server-side:

```
Smeta
→ Fakt
→ F2
→ Qoldiq
```

hisoblanadi.

F2/Fakt mutationlari alohida domain transactionlar:

```
f2_yarat
f2_yopish
fakt_yarat
fakt_tasdiqlash
```

kabi bo'ladi.

Sheets ichida katta smetani qayta-qayta scan qilish tugatiladi.

---

## 14. FINANCE

Bitta `price` hamma narsani anglatmasin.

Alohida tushunchalar:

```
estimate_price
contract_price
supplier_price
purchase_price
invoice_amount
actual_cost
market_price
sales_price
payment_amount
```

Zanjir:

```
Contract
→ Invoice
→ Payment
```

aloqida kuzatiladi.

---

## 15. PROCUREMENT + WAREHOUSE

Estimate

→ Material Requirement

→ Purchase

→ Supplier

→ Invoice

→ Warehouse Receipt

→ Stock

→ Issue/Consumption

→ Fact

Tizim quyidagi savolga bitta graphdan javob berishi kerak:

Qancha kerak edi → qancha olindi → qancha keldi → qancha ishlatildi → qancha qoldi?

## 16. PROJECT/BIM/TAKEOFF

Kelajakdagi integration:

DWG

PDF

IFC

RVT

Excel

→ Extraction

→ Normalization

→ Quantity Takeoff

→ Resource Mapping

→ Estimate Comparison

Masalan:

Project quantity = 1 284 m<sup>3</sup>

Estimate = 1 190 m<sup>3</sup>

Difference = +94 m<sup>3</sup>

Source/provenance saqlanadi.

## 17. DIDOX / EDO

Core integration interface:

EDO Adapter

Uzbekistan implementation:

Didox

Bogʻlanadigan obyektlar:

- counterparty
- contract
- invoice/EHF
- document state
- amount
- VAT
- external ID

Import idempotent boʻladi.

---

## 18. BANK

Core:

Bank Adapter Interface

Bank transaction:

```
account
transaction_id
date
amount
currency
counterparty
description
reference
```

Reconciliation:

```
Bank Transaction
→ Candidate Match
→ Invoice / Payment / Contract
→ Reconciled / Unmatched
```

AI candidate match beradi, final accounting state deterministic rule/approval bilan yoziladi.

---

## 19. SALES / REAL ESTATE

```
Project
→ Building
→ Section
→ Floor
→ Unit
→ Reservation
→ Sales Contract
→ Payment Schedule
→ Payment
```

Status:

```
available
reserved
sold
cancelled
```

Sales project profitability bilan bog'lanadi.

---

## 20. AI DATA LAYER

Hozirgi AI retrieval yondashuvi quyidagiga evolyutsiya qiladi:

```
USER QUESTION
→ AI ORCHESTRATOR
→ INTENT
→ DATA TOOL / SQL TOOL / DOCUMENT TOOL
→ SUPABASE
→ VERIFIED EVIDENCE
→ AI ANSWER
```

Butun database'ni promptga yuborish taqiqlanadi.

---

## 21. AI SEMANTIC LAYER

AI quyidagilarni canonical entityga resolve qila olishi kerak:

```
M200
M-200
M200
Бетон М200
Бетон марки 200
```

→ bitta material entity.

Semantic metadata:

```
canonical_id
aliases
unit
category
country
standard
relations
```

---

## 22. AI SAVOLGA JAVOB

Savol:

M200 beton qancha olingan?

Tizim:

```
material = M200
metric = purchased_quantity
scope = project/object
→ database tool
→ deterministic aggregation
→ evidence
→ LLM explanation
```

Javob faqat database daliliga asoslanadi.

Agar exact topilmasa:

```
exact
→ alias
→ semantic
→ related table
→ documents
```

tekshirilgandan keyin “topilmadi” deyiladi.

---

## 23. AI AGENTLAR

Bitta ulkan agent emas.



## **Orchestrator**

Boshqaradi va kerakli agent/toolni tanlaydi.

## **Estimate Agent**

Smeta, classification, pricing, variance.

## **F2/Fact Agent**

Fakt, F2, qoldiq, nakopitelniy.

## **Procurement Agent**

Material ehtiyoji, supplier, narx, reorder.

## **Warehouse Agent**

Qoldiq, kirim, chiqim, deficit.

## **Finance Agent**

Cashflow, AP/AR, bank, Didox, payment.

## **Document Agent**

OCR, invoice, contract, Drive.

## **Project/BIM Agent**

Takeoff va loyiha-smeta taqqoslash.

## **Sales Agent**

Unit, reservation, sales, payments.

## **Audit Agent**

Nomuvofiqliklar va risk.

## **Monitoring Agent**

Daily/system/project anomalies.

## **Performance Agent**

Slow query/API/system analysis.

## **Code/Architecture Agent**

GitHub, test, regression, improvement proposal.

---

## 24. AGENT TOOL MODEL

Read tools:

```
get_project
get_object
get_estimate_summary
get_material_summary
get_purchase_summary
get_fact_summary
get_f2_summary
get_balance
get_price_history
search_documents
```

Controlled write tools:

```
create_f2_draft
create_purchase_draft
create_notification
create_document_record
```

Agentga:

```
generic SQL
generic DELETE
unrestricted filesystem
unrestricted production deploy
```

berilmaydi.

---

## 25. AGENT AVTONOMIYASI

### L1 — READ

O'qiydi, tahlil qiladi, xabar beradi.

### L2 — CONTROLLED ACTION

Draft yoki ruxsatlangan mutation.

### L3 — AUTONOMOUS

Schedule orqali o'zi ishlaydi.

Real tashqi ta'sirli ishlar approval/HITL bilan.

## 26. SELF-IMPROVEMENT

OBSERVE

→ DETECT

→ ANALYZE

→ PROPOSE

→ SANDBOX

→ TEST

→ COMPARE

→ APPROVAL

→ DEPLOY

→ MONITOR

Agent production kodini o'zicha almashtirmaydi.

Masalan Performance Agent slow query topadi, candidate index/test beradi, lekin deploy approvaldan o'tadi.

---

## 27. CONTROL CENTER

Dastlab alohida frontend shart emas.

Mavjud frontend ichida:

/admin

Keyinchalik kerak bo'lsa alohida subdomain.

Control Center:

Companies

Users

Projects

Integrations

AI

Agents

Jobs

Sync

Conflicts

Errors

Performance

Documents

Audit

System Health

---

## 28. GLOBAL ARXITEKTURA

Core global.

Country-specific layer:

```
Uzbekistan Pack
UAE Pack
Saudi Pack
USA Pack
EU Pack
UK Pack
...
```

Country pack:

- tax
- accounting rules
- EDO
- banking
- construction standards
- terminology
- units
- currency
- government integrations
- document requirements
- signatures

**Uzbekistan-specific qoidalar core'ga hard-code qilinmaydi.**

Masalan:

```
Core: Progress Certification
UZ adapter: F2
Other country: local equivalent
```

---

## 29. LOCALIZATION

```
language
date_format
number_format
currency
timezone
```

Multi-language va terminology dictionary.

---

## 30. MULTI-CURRENCY

Alohida:

```
company_currency
project_currency
contract_currency
transaction_currency
reporting_currency
```

Exchange-rate tarixiy snapshot saqlanadi.

Eski invoice keyingi kurs o'zgargani uchun o'zgarmaydi.

---

## 31. SECURITY — OXIRGI KATTA BOSQICH

Hozirdanoq faqat architecture-ready talablar:

- secret source code'da emas;
- service role clientga chiqmaydi;
- generic SQL endpoint yo'q;
- domain RPC;
- immutable ID;
- version;
- audit;
- operation\_id;
- source/origin.

Final security stage:

1. RLS
2. full RBAC/ABAC
3. SSO/MFA
4. tenant isolation
5. rate limiting
6. secret rotation
7. webhook signature
8. audit integrity
9. backup/restore
10. security regression

11. penetration/security testing

12. data residency/privacy

---

## **32. MIGRATION ROADMAP**

### **Phase 0 — Baseline**

Tizim\_01 snapshot + golden datasets + rollback.

### **Phase 1 — T2 Core**

Raw → normalize → classify → price → rollup.

### **Phase 2 — Real Data**

Amfiteatr 4,937 qatorli golden migration.

### **Phase 3 — Version/Conflict**

Version + audit + idempotency.

### **Phase 4 — Frontend Full Client**

Read/write + conflict.

### **Phase 5 — Sheets Full Client**

Bidirectional sync + hidden ID/version + conflict.

### **Phase 6 — F2/Fact/Nakopitelniy**

Financial core Postgres'da.

### **Phase 7 — Procurement/Warehouse**

### **Phase 8 — Documents/Drive**

### **Phase 9 — Didox/Bank**

### **Phase 10 — Sales**

### **Phase 11 — BIM/Takeoff**

### **Phase 12 — AI Data Layer**

### **Phase 13 — Agents**

### **Phase 14 — Self Improvement**

## Phase 15 — Security Hardening

## Phase 16 — Global Country Packs

## Phase 17 — Production Cutover

---

### 33. GOLDEN TEST

Kamida:

- kichik dataset
- o'rta dataset
- Amfiteatr 4,937 qator
- 20k+ stress

Comparison:

```
row count
hierarchy
quantity
unit
price
category
total
unpriced
negative rows
Fact
F2
remaining
```

Tizim\_01 ↔ Tizim\_02.

Farq yopilmasdan keyingi critical stage yo'q.

---

### 34. CLAUDE CODE PROTOCOL

Har task:

1. `00_BOSH_QONUN.md` o'qilsin.
2. Tizim\_02 canonical docs o'qilsin.
3. Existing RPC/function/adapter qidirilsin.
4. Dependency/data-flow aniqlansin.
5. Minimal patch plan.
6. Kod.

7. Test.
8. Regression.
9. Reconciliation.
10. Change/risk/rollback report.

Har katta taskdan keyin Claude quyidagilarni beradi:

```
What changed
Why
Files changed
Tests
Reconciliation
Performance
Risks
Rollback
Next single task
```

---

## 35. CLAUDE QILMASLIGI KERAK

- Tizim\_01ni buzmaslik.
- Existing logicni topmasdan duplicate logic yozmaslik.
- Business logicni GAS/Frontend/SQLda uch marta yozmaslik.
- Sheet row numberni primary key qilmaslik.
- Versionni chetlab o'tmaslik.
- Financial LWW ishlatmaslik.
- Fake data qo'shmaslik.
- Generic SQL endpoint yaratmaslik.
- Alga financial truth ixtiro qildirmaslik.
- 20k rowsni browserga birdan yubormaslik.
- Benchmark qilmasdan performance va'dasi bermaslik.
- Migration discrepancy qolganida keyingi critical featurega sakramaslik.
- Security-ready contractlarni buzmaslik.
- Full security hardeningni hozirgi developmentni keraksiz og'irlashtiradigan darajada oldinga surmaslik.

---

## 36. BIRINCHI ISHCHI TASKLAR

### TASK 1

Hozirgi T2/Supabase/Sheets bridge/F2/Fact/AI qatlamining dependency graphini chiqarish. Kod o'zgartirilmasin.



## **TASK 2**

Amfiteatr 4,937 qator uchun golden import/reconciliation harness.

## **TASK 3**

Tizim\_01 vs Tizim\_02 differential report.

## **TASK 4**

Version/conflict contractni yakunlash.

## **TASK 5**

Frontend full-client edit.

## **TASK 6**

Sheets full-client bidirectional sync.

## **TASK 7**

F2/Fact/Nakopitelniy canonical SQL engine.

## **TASK 8**

1k/5k/20k/50k benchmark.

## **TASK 9**

Document/Drive integration.

## **TASK 10**

AI Data Layer → tools → orchestrator.

---

## **37. DEFINITION OF DONE**

Tizim\_02:

- Supabase canonical
- Frontend full client
- Sheets full client
- Drive integrated
- ID/version/conflict
- Audit
- Idempotency
- F2/Fact/Nakopitelniy server-side
- Procurement

- Warehouse
- Finance foundation
- 4,937 golden reconciliation
- 20k benchmark
- AI data layer
- Agents
- Control Center
- rollback
- security hardening
- global country abstraction

bitmaguncha “final production platform” deyilmaydi.

---

## 38. NORTH STAR

Foydalanuvchi:

Sheets  
Web  
Mobile  
Telegram

istalgan klientdan ishlaydi.

Supabase bitta markaziy haqiqat.

Tizim foydalanuvchining savoliga real data orqali javob beradi:

“M200 qancha olingan?”

“Qancha qoldi?”

“Kimdan olindi?”

“Qancha to‘ladik?”

“F2 bilan fakt mosmi?”

“Qaysi material yetishmayapti?”

“Loyihada qaysi joyda variance bor?”

“Bugun nima qilish kerak?”

Agentlar esa kuzatadi, tahlil qiladi, tavsiya beradi va ruxsatlangan ishlarni avtomatlashtiradi.

Global mahsulotning konsepti:

```
ONE CORE
+
MULTIPLE CLIENTS
+
MULTIPLE INTEGRATIONS
+
COUNTRY PACKS
+
AI INTELLIGENCE
+
CONTROLLED AUTONOMY
```

""""

## Save Markdown

```
md_path =
Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.md")
md_path.write_text(md, encoding="utf-8")
```

## Save DOCX

```
docx_path =
Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.docx")
doc = Document()
sec = doc.sections[0]
sec.top_margin = Inches(0.65); sec.bottom_margin = Inches(0.65)
sec.left_margin = Inches(0.7); sec.right_margin = Inches(0.7)

p = doc.add_paragraph()
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
r = p.add_run("TIZIM_02 — GLOBAL CONSTRUCTION OS")
r.bold = True; r.font.size = Pt(22)
p = doc.add_paragraph()
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
r = p.add_run("Yakuniy arxitektura va rivojlantirish rejasi")
r.italic = True; r.font.size = Pt(12)
```

```

for block in md.split("\n\n"):
 block = block.strip()
 if not block: continue
 if block.startswith("# "):
 continue
 if block.startswith("## "):
 doc.add_heading(block[3:], level=1); continue
 if block.startswith("### "):
 doc.add_heading(block[4:], level=2); continue
 if block.startswith(" ") and block.endswith(" "):
 p = doc.add_paragraph()
 rr = p.add_run(block[3:-3].strip())
 rr.font.name = "Consolas"; rr.font.size = Pt(8)
 continue
 for line in block.splitlines():
 line=line.strip()
 if not line: continue
 if line.startswith("- "):
 doc.add_paragraph(line[2:], style="List Bullet")
 elif re.match(r"^\d+\s", line):
 doc.add_paragraph(re.sub(r"^\d+\s*", "", line), style="List Number")
 else:
 doc.add_paragraph(line)

for s in doc.sections:
 s.header.paragraphs[0].text = "TIZIM_02 — GLOBAL CONSTRUCTION OS | MASTER ARCHITECTURE"
 s.footer.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
 s.footer.paragraphs[0].text = "Supabase Core • Sheets + Web Full Clients • Drive • AI/Agents • Global Architecture"

doc.save(docx_path)
print(f'{md_path}\n{docx_path}')

```

```

NameError Traceback (most recent call last)
Cell In[1], line 1267
 1263 line=line.strip()
 1264 if not line: continue
 1265 if line.startswith("- "):
 1266 doc.add_paragraph(line[2:], style="List Bullet")
-> 1267 elif re.match(r"^\d+\s", line):
 1268 doc.add_paragraph(re.sub(r"^\d+\s*", "", line), style="List Number")

```

```
1269 else:
1270 doc.add_paragraph(line)
```

NameError: name 're' is not defined

## Analysis errored

### Python

```
import re
from pathlib import Path
from docx import Document
from docx.shared import Inches, Pt
from docx.enum.text import WD_ALIGN_PARAGRAPH

md_path = Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.md")
docx_path = Path("/mnt/data/TIZIM_02_GLOBAL_CONSTRUCTION_OS_ARXITEKTURA_REJA.docx")

The full Markdown content from the prior generation is already present in md_path only
if the previous write completed. Recreate it from the same source variable when available
if 'md' not in globals():
 raise RuntimeError("Master text is not available in the current execution context.")

md_path.write_text(md, encoding="utf-8")

doc = Document()
sec = doc.sections[0]
sec.top_margin = Inches(0.65); sec.bottom_margin = Inches(0.65)
sec.left_margin = Inches(0.7); sec.right_margin = Inches(0.7)

p = doc.add_paragraph()
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
r = p.add_run("TIZIM_02 - GLOBAL CONSTRUCTION OS")
r.bold = True; r.font.size = Pt(22)
p = doc.add_paragraph()
p.alignment = WD_ALIGN_PARAGRAPH.CENTER
r = p.add_run("Yakuniy arxitektura va rivojlantirish rejasi")
r.italic = True; r.font.size = Pt(12)

for block in md.split("\n\n"):
 block = block.strip()
 if not block:
 continue
 if block.startswith("# "):
 continue
 if block.startswith("## "):
 doc.add_heading(block[3:], level=1)
 continue
 if block.startswith("### "):
```

```

 doc.add_heading(block[4:], level=2)
 continue
 if block.startswith("```") and block.endswith("```"):
 p = doc.add_paragraph()
 rr = p.add_run(block[3:-3].strip())
 rr.font.name = "Consolas"; rr.font.size = Pt(8)
 continue
 for line in block.splitlines():
 line = line.strip()
 if not line:
 continue
 if line.startswith("- "):
 doc.add_paragraph(line[2:], style="List Bullet")
 elif re.match(r"^\d+\.\s", line):
 doc.add_paragraph(re.sub(r"^\d+\.\s*", "", line), style="List Number")
 else:
 doc.add_paragraph(line)

for s in doc.sections:
 s.header.paragraphs[0].text = "TIZIM_02 - GLOBAL CONSTRUCTION OS | MASTER ARCHITECTURE"
 s.footer.paragraphs[0].alignment = WD_ALIGN_PARAGRAPH.CENTER
 s.footer.paragraphs[0].text = "Supabase Core • Sheets + Web Full Clients • Drive • AI"

doc.save(docx_path)
print(f"CREATED {md_path}\n{docx_path}")

```

It seems like I can't do more advanced data analysis right now. Please try again later. 

You've reached your data analysis limit. Upgrade to ChatGPT Plus or try again tomorrow after 8:38 AM.

[Rejoin Plus](#)